Based on claude code leaks, how does claude code manage and maintain context to prevent context rot and context window overflow?

provide a comprehensive description of techniques for context management in claude code from official documentation and source code leaks

make it technical and descriptive. the goal is to design a similar system using langgraph library

Claude Code manages context by treating it as layered, policy-driven state (DAG history, skills, memory, tools) and continuously projecting only a compact, high-signal slice into the model's token window, with aggressive compaction and isolation via subagents to avoid context rot and overflow. A similar system in LangGraph should explicitly model session history as a DAG, add compaction and skill/memory managers as graph nodes, and assemble each turn's prompt from these layers under a token budget rather than pushing a raw rolling chat log.[1] [2] [3]

What "context" is in Claude Code

Claude Code distinguishes live model context (what goes into the API call) from durable state (what is persisted and can be re-projected later).[2] [1]

- Live context per call includes: system prompt + tool schemas, skill metadata, CLAUDE.md and a slice of MEMORY.md, selected conversation messages and summaries, and recent file/command outputs, plus any required thinking block for tool use.[4] [1] [2]
- Durable state includes: JSONL session files, checkpoints, the filesystem (repo), CLAUDE.md / MEMORY.md / skills, and memory/topic files; these are mostly *outside* the token window and are pulled in on demand.[5] [1] [2]

The `/context` view and writeups show breakdowns like: system prompt, built-in tools, skills descriptions, free space, and an "autocompact buffer" reserved so compaction can run without hitting the hard limit.[6] [2]

Session DAG and history storage

On disk, each project has JSONL session files under `~/.claude/projects/<project-slug>/<session-uuid>.jsonl`, where each line is a mini-event object.[7]

Key fields in each JSON object:[7]

- `uuid`: unique message id.
- `parentUuid`: parent message id (or `null` for root).

- `type`: `user`, `assistant`, `summary`, etc.
- `message.role` / `message.content`: what the model saw.
- Metadata: `sessionId`, `cwd`, `gitBranch`, version, thinkingMetadata, etc.

`parentUuid` turns this into a directed acyclic graph (DAG) of messages, essentially "commits", so each conversation branch is reconstructed by walking parent links from a *leaf* (a `uuid` that is never used as any other node's `parentUuid`).[7]

On `--resume`, Claude Code reads *all* project JSONL files, parses them into a flat list, rebuilds the global DAG, finds leaf messages, and treats each leaf as a separate conversation tip for the resume menu. Summaries are separate `type:"summary"` entries carrying `leafUuid` references that title or summarize a conversation.[8] [7]

LangGraph analogue

- Persist messages in a store keyed by `uuid` with `parent_id`, `type`, `role`, `content`, and metadata.
- Maintain the "current leaf id" in LangGraph state.
- Build a `BuildContextSlice` node that walks ancestors from the leaf, decides which nodes/summaries to include, and emits a linear list of messages for the LLM.
- Forking is just creating a new message whose `parent_id` points at an older node.

Automatic and manual compaction

Auto-compaction behavior

Docs describe a multi-step strategy as context fills:[1] [2]

1. Track tokens and remaining budget (with reserved autocompact buffer).[4] [6]
2. First drop old **tool outputs** and very verbose logs, as they are cheap to recompute or summarize.[2] [1]
3. If still near the limit, **summarize older conversation** into compact synthetic messages preserving user goals, key decisions, and critical code snippets.
4. Obey compaction hints from CLAUDE.md (e.g. "when compacting, always preserve modified file list and test commands").[2]

Separately, the Claude API supports server-side compaction for long conversations, which Claude Code uses as a primary strategy for avoiding window overflow in agentic workflows.[3] [4]

`/compact` command and prompt

The `/compact` command triggers summarization on demand and accepts an optional focus string (e.g. `/compact focus on the API changes`). Community posts show the internal prompt template, which asks Claude to produce a structured summary with sections like:[9] [10] [2]

- Primary request and intent (all user requests in detail).

- Key technical concepts.
- Files and code sections (including why they matter and important snippets).
- Problems solved and ongoing troubleshooting.
- Pending tasks.
- Current work (emphasis on most recent messages and files).
- Optional next step.[10]

The harness sends that over the current strip of messages + outputs, gets back the structured summary, inserts it into history, and then prefers that summary message in future context slices instead of dozens of raw messages.[10] [2]

Rewind-based partial compaction

`Esc Esc` or `/rewind` lets you pick a checkpoint message and "Summarize from here", compacting *only* messages after that point. Under the hood, that is equivalent to inserting a `summary` node whose parent is the checkpoint and marking the intervening nodes as not normally included in future prompts while still keeping them in the DAG for deep recall.[7] [2]

Leak-derived detail: multiple compaction modes

Analyses of the leaked code report **five distinct compaction strategies**, handling conversation, tool results, and memory differently, and an 8-phase compaction/processing pipeline overall. The public docs already hint at this with separate notions of "server-side compaction", "tool result clearing", and "thinking block clearing", which are composed depending on which part of state is dominating the window.[3] [4] [1]

LangGraph analogue

Implement compaction as explicit nodes:

- `AutoCompactIfNeeded` – called before each LLM step; estimates token usage for the planned slice and, if above threshold, does:
  - "tool-result clearing" – drop or aggressively summarize old tool results;
  - "conversation summarization" – call a summarizer model with a `/compact`-style prompt and insert a `summary` node at a chosen checkpoint.
- `ManualCompact` – triggered when the user sends `/compact` with focus instructions; same summarizer but with explicit user focus.
- `PartialCompactFromCheckpoint` – compact from a given message id, aliasing the sequence to a summary node.

Then, ensure the context builder prefers summaries when present and only unfolds details on explicit user request.

Skills and progressive disclosure

Claude Code's **Skills** are structured instruction bundles with optional scripts and references, and they are explicitly designed to reduce baseline context usage.[11] [5]

Behavior from the skills docs:[5]

- **Metadata layer:** At startup, Claude sees `name` and `description` for each skill (used to decide when to invoke it). Descriptions share a fixed *character budget* (~1% of window, fallback 8000 chars) and are truncated to 250 chars each in the listing.[5]
- **Instruction layer:** The `SKILL.md` body is loaded into context only when the skill is invoked (automatically or via `/skill-name`).
- **Resource layer:** Supporting markdown files are read only if referenced; executable scripts are run via Bash and *only their output* is injected, not the script source.[5]

Frontmatter controls when skills enter context at all:[5]

- `disable-model-invocation: true` – prevents Claude from auto-invoking the skill and can keep even its description out of context until manually triggered.
- `user-invocable: false` – hides a skill from user `/` menus while allowing Claude to call it as background reference.
- `context: fork` + `agent: ...` – runs the skill in a forked subagent with its own context window.

This architecture is a direct response to context rot from monolithic CLAUDE.md files: long, specialized instructions move into skills, and only the metadata remains resident in baseline context.[11] [2] [5]

LangGraph analogue

- Maintain a skill registry (metadata only) in a separate store and optionally expose a "SkillSearch" tool.
- Load full skill content into context only when the LLM selects or the user commands it.
- Treat large references as files accessible via tools, not pre-injected text.
- Make `disable-model-invocation` and `context: fork` first-class flags in your orchestration – they decide whether a skill's prompt is appended to the main strip or run in a subgraph.

Subagents and forked contexts

Subagents are configured via `.claude/agents/*.md` and can be targeted by skills. Key properties: [1] [2] [5]

- Each subagent has its own system prompt, allowed tools, and *independent conversation/context window*.
- When a skill has `context: fork`, its SKILL.md becomes the prompt for the subagent instead of being appended to the main context.[5]

- The subagent runs, uses tools, reads many files, and then returns a **summary** back to the main session, often plus artifacts (diffs, files, PRs).

Leak analyses also point out that Claude Code ties subagents into a **prompt caching / KV cache** scheme, so the shared prefix (system prompt + tools + shared repo info) is reused across agents without re-charging tokens. This makes "fork-join" patterns (parallel codebase exploration or multi-agent refactoring) much cheaper.[3]

LangGraph analogue

- Use nested graphs: a `SubagentRunner` node invokes a second LangGraph app with its own state and history store.
- Give the subagent only a task spec + minimal background (e.g. CLAUDE.md-equivalent) and no main conversation by default.
- Have subagent return a summarized result node for the main graph to insert into history.
- If your model/API supports prompt caching, keep system/tool prompts stable to maximize cache hits.

Memory: CLAUDE.md, MEMORY.md, autoDream

Docs describe two main textual memories:[1] [2]

- CLAUDE.md – persistent project instructions read at every session start; should be short and contain only widely-applicable rules and workflows.
- MEMORY.md – automatically maintained file where Claude stores observed preferences, patterns, and facts; only the first 200 lines or 25 KB (whichever comes first) are loaded per session to bound cost.[1]

CLAUDE.md can `@import` other files (README, extra docs, per-directory CLAUDE.md), but the best-practices docs strongly recommend keeping it concise to avoid drowning important rules in noise.[2]

From the leak summaries:[3]

- Memory has **3 layers**:
  1. `MEMORY.md` as a high-level index.
  2. Topic-specific memory files loaded on demand.
  3. Full session transcripts that can be searched when necessary.
- An offline **autoDream** process periodically merges and deduplicates memories, attempting to remove contradictions and stale entries.[3]

This further decouples long-term knowledge from the live context window.

LangGraph analogue

- Use a light "memory index" document always available at startup and searchable within the orchestrator (not necessarily injected whole).
- Store per-topic memories and past transcripts in a DB or vector store.
- Expose tools like `SearchMemory` or `SearchTranscripts` that the model can call to pull in specific snippets only when needed.
- Optionally run offline background jobs to condense older memory shards.

Tool and repo context minimization

Claude Code aggressively treats the project repo and external tools as **external memory** rather than content to preload.[12] [1] [3]

Patterns from docs and leak-driven writeups:[2] [1] [3]

- It automatically surfaces git state (branch, uncommitted changes, recent commits) when relevant, but otherwise reads only the specific files it needs via tools.
- It implements **file read deduplication** and **tool result sampling**: repeated reads of the same file are deduped or replaced by summaries, and large logs are truncated and then summarized instead of being shoved in raw.[3]
- It encourages using CLIs like `gh` for GitHub integrations and then injects only the CLI output into context (issues, PR diffs, comments), not entire API docs or schemas.[12] [2]

This supports the general compaction rule: drop or summarize tool outputs first, before giving up user instructions and code.[1] [2] [3]

LangGraph analogue

- Put a `ToolResultFilter` node between raw tool invocations and the LLM; bound tool-result tokens per turn and summarize or chunk as needed.
- Cache per-file summaries so the LLM deals mostly with structured descriptions + selective snippets.
- Expose CLIs and APIs as tools that produce concise JSON structures; let the model reason over structured results, not giant text dumps.

Model-side context features: thinking tokens and budgets

Claude's **extended thinking** (chain-of-thought) counts toward the context window, but prior thinking blocks are automatically excluded from subsequent turns' context calculation (except in the one place they must be echoed: the tool-result turn). That is, once a tool call cycle is complete, the API drops the previous `thinking` block from what the model "sees" next turn while still billing its tokens once.[4]

Newer models (Sonnet 4.6, Opus 4.6) are **explicitly context-aware**: they get a `<budget:token_budget>...` tag and periodic `<system_warning>` updates like `Token usage:`

`35000/1000000; 965000 remaining` to guide how verbose they should be and when to ask for compaction.[4]

Claude Code surfaces this to users with `/context` and context-percentage indicators in the CLI/IDE; issues mention that quality often degrades beyond ~40–50% usage, even though the hard limit is much higher.[13] [14]

LangGraph analogue

- Use a token estimation function to track budget and adopt soft thresholds (e.g. 40%: suggest compaction; 70%: auto-compact; 90%: block new work until compaction/clear).
- Optionally inject your own "budget warnings" as system messages for the LLM.
- Provide commands like `/context`, `/clear`, `/btw` implemented as special graph entry points —`/btw` should execute a one-off branch that does not write to main history.

Attachment/history filtering: `db8`

A Reddit analysis of the leaked TS source notes a function `db8` which determines what events/attachments are written into the session JSONL files. For non-Anthropic users, it reportedly drops **all attachment-type messages**, including `deferred_tools_delta` entries that track which tools have been introduced to the model.[15]

The bug: stripping those deltas means resumed sessions may lack information about previous tool introductions, forcing the harness to reintroduce tools and potentially causing inconsistent behavior or extra context usage.[15]

Conceptually, this is a **history filter layer** between live events and persisted history, deciding which message sub-components are durable and which are ephemeral.

LangGraph analogue

- Introduce a `HistoryFilter` node that decides which parts of each turn (attachments, artifacts, tool traces) are persisted vs kept only in ephemeral state.
- Make the filtering policy explicit and versioned (e.g. different behavior for internal vs external or low-trust vs high-trust environments).
- Be careful not to filter out metadata that is needed for correct reconstruction of context (the `deferred_tools_delta` example is exactly what happens when the filter is too aggressive).

User-level context controls

Claude Code exposes context management directly to users:[12] [2] [1]

- `/clear` – reset the live context (fresh window for the same project).
- `/context` – show current usage by category, including system prompt, tools, skills, free space, and autocompact buffer.[6]
- `/compact [focus]` – trigger immediate summarization with focus instructions.[9] [2]

- `/rewind` / `Esc Esc` – restore earlier checkpoints or summarize from a chosen message.[2]
- `/btw` – run transient queries whose answers never enter the conversation history, avoiding context growth.[2]

VS Code issues discuss surfacing a persistent context percentage indicator and making the threshold configurable; right now the indicator appears at 80%, which is more of an "emergency light" than a proactive tool, while power users track the "<40–50%" rule manually via `/context`.[14] [13]

LangGraph analogue

You can map these directly to graph entrypoints or control messages:

- `/clear` → reset the context buffer and compaction state while preserving durable DAG history.
- `/context` → run a diagnostics node that computes a breakdown from the planner and displays it.
- `/btw` → execute a minimal graph that routes the prompt through tools/LLM but never writes to history.
- `/rewind` → update the current leaf pointer to a previous node; optionally call `PartialCompactFromCheckpoint`.

You can find all of this, with diagrams and a more formal breakdown plus concrete LangGraph design suggestions, in the attached report. It covers the session DAG, compaction modes, skills and subagents, memory layers, and a mapping of each Claude Code pattern to specific LangGraph nodes and stores.

⁂

1. https://code.claude.com/docs/en/how-claude-code-works
2. https://piebald.ai/blog/messages-as-commits-claude-codes-git-like-dag-of-conversations
3. https://code.claude.com/docs/en/best-practices
4. https://www.latent.space/p/ainews-the-claude-code-source-leak
5. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview
6. https://www.reddit.com/r/ClaudeAI/comments/1s8zxt4/thanks_to_the_leaked_source_code_for_claude_code/
7. https://platform.claude.com/docs/en/build-with-claude/context-windows
8. https://www.linkedin.com/posts/kylelawrencejensen_the-context-command-will-show-you-how-much-activity-7392655875334709248-Vxxv
9. https://github.com/anthropics/claude-code/issues/5135
10. https://github.com/anthropics/claude-code/issues/21781
11. https://github.com/anthropics/claude-code/issues/18456
12. https://www.reddit.com/r/ClaudeAI/comments/1jr52qj/here_is_claude_codes_compact_prompt/

13. https://www.linkedin.com/posts/kylemickey_how-to-switch-between-tasks-in-claude-code-activity-7392963357160501248-6Ngk
14. https://www.linkedin.com/pulse/claude-code-skills-how-ai-loads-knowledge-demand-sergey-smirnov-idygc
15. https://www.cometapi.com/managing-claude-codes-context/
16. https://www.damiangalarza.com/posts/2025-12-08-understanding-claude-code-context-window/
17. https://theouterloop.substack.com/p/a-few-fun-things-you-can-do-with
18. https://cybernews.com/security/anthropic-claude-code-source-leak/
19. https://www.aipaths.academy/en/docs/002_claude-context-window
20. https://www.reddit.com/r/ClaudeAI/comments/1r01129/context_management_is_everything_a_concise_guide/
21. https://www.reddit.com/r/ClaudeAI/comments/1s9d9j9/claude_code_source_leak_megathread/
22. https://cybernews.com/tech/claude-code-leak-spawns-fastest-github-repo/
23. https://www.youtube.com/watch?v=lt-wKINAMuw
24. https://code.claude.com/docs/en/vs-code
25. https://www.reddit.com/r/ClaudeCode/comments/1pa0s0h/is_there_a_way_to_have_claude_code_search_the/
26. https://github.com/anthropics/claude-code/issues/12990
27. https://db8.nl/en/blog/joomlacamp-2026-in-essen-germany
28. https://github.com/RooCodeInc/Roo-Code/discussions/1418
29. https://www.youtube.com/watch?v=Ccot5rRT9CY
30. https://code.claude.com/docs/en/skills